		List Name	Type	Member of class & its declaration
		IMPORTANT **All the following lists are lists of POINTERS not objects** **For any DS in the project, you msut use the DS classes given in the labs (or derived from them)** **not allowed to use your own classes or any other classes like STL classes** **Bonus handling not included**		
	1	**ACTIONS_LIST**	**Queue**	**Restaurant** *LinkedQueue<action*> List_name;*
Pending Orders	2	**PEND_ODG**	**Queue**	**Restaurant** *LinkedQueue<order*> List_name;*
	3	**PEND_ODN**	**Queue**	**Restaurant** *LinkedQueue<order*> List_name;*
	4	**PEND_OT**	**Queue**	**Restaurant** *LinkedQueue<order*> List_name;*
	5	**PEND_OVN**	**Queue**	**Restaurant** *LinkedQueue<order*> List_name;*
	6	**PEND_OVC**	**a class derived from Queue** *class **Pend_OVC** : public LinkedQueue<order*>* *{ //add "CancelOrder(ID)" function }*	**Restaurant** *Pend_OVC List_name;*
	7	**PEND_OVG**	**PriQueue**	**Restaurant** *PriQueue<order*> List_name;*
Free Chefs	8	**Free_CS**	**Queue**	**Restaurant** *LinkedQueue<chef*> List_name;*
	9	**Free_CN**	**Queue**	**Restaurant** *LinkedQueue<chef*> List_name;*
	10	**Cancelled_orders**	**Queue** **Stack would be accepted as well**	**Restaurant** *LinkedQueue<order*> List_name;*
	11	**Finished_orders**	**Stack**	**Restaurant** *LinkedQueue<order*> List_name;*

Reason (Verbal) **(Actual justification should include big O analysis)**			
actions are added and picked to/from list according to action timestep Notice that this list is loaded from the input file which is already sorted by action timestep			
FCFS: These orders are assigned to Chefs in the order they arrive.			
FCFS: These orders are assigned to Chefs in the order they arrive.			
FCFS: These orders are assigned to Chefs in the order they arrive.			
FCFS: These orders are assigned to Chefs in the order they arrive.			
Derived: These orders are FCFS but require a CancelOrder function to remove them if a cancellation action (X) occurs.			
orders leave this list according to a priority criteria (as stated in the doc)			
Assigend based on FCFS criteria			
Assigend based on FCFS criteria			
They are normally printed on the screen in the same order of insertion (cancellation) Stack is accrpted too as printing order is not clear in the doc			
on the o/p screen and in the o/p file, recenly finished orders are printed first			

		List Name	Type	Member of class & its declaration
	12	Cooking_Orders	**a class derived from PriQueue** *class **Cook_Ords** : public PriQueue<order*>* *{ //add "CancelOrder(ID)" function }*	**Restaurant** *Cook_Ords List_name;*
Ready Orders	13	RDY_OT	Queue	**Restaurant** *LinkedQueue<order*> List_name;*
	14	RDY_OV	**a class derived from Queue** *class **RDY_OV** : public LinkedQueue<order*>* *{ //add "CancelOrder(ID)" function }*	**Restaurant** *RDY_OV List_name;*
	15	RDY_OD	Queue	**Restaurant** *LinkedQueue<order*> List_name;*
	16	InServ_Orders	PriQueue	**Restaurant** *PriQueue<order*> List_name;*
Scooters	17	Free_Scooters	PriQueue	**Restaurant** *PriQueue<scooter*> List_name;*
	18	Back_Scooters	PriQueue	**Restaurant** *PriQueue<scooter*> List_name;*
	19	Maint_Scooters	Queue	**Restaurant** *LinkedQueue<scooter*> List_name;*
Tables	20	Free_Tables	**a class derived from PriQueue** *class **Fit_Tables** : public PriQueue<table*>* *{ //add "getBest()" function }*	**Restaurant** *Fit_Tables Free_Tables;*
	21	Busy_Sharable	**a class derived from PriQueue** *class **Fit_Tables** : public PriQueue<table*>* *{ //add "getBest()" function }*	**Restaurant** *Fit_Tables Busy_Sharable;*

Reason (Verbal) **(Actual justification should include big O analysis)**			
- PriQ: orders leave the list depending on prapration time - Derived: to support CancelOrder function to remove them if a cancellation action (X) occurs. **It is also acceptable to have 2 lists here (one for Cooking OVC and one for all other types)**			
They are picked by the customer after one timestep in the same order they are ready			
- They are picked for service in the same order they are ready - Derived: to support order cancellation			
They are picked for service in the same order they are ready			
orders leave the list depending on service duration			
Assigend based on the shortest distance			
Leaves the list based on the distance they cut back to the restaurant			
All scooters have the same maintenance time So they leave the list the same order they enter it (FCFS)			
PriQ: as table are prioritrized by least no. of avail seats (Best fit) Derived: The item at head of the priQ may not fit to the current order So you should add a function that returns the best-fit table for that order			
PriQ: as table are prioritrized by least no. of avail seats (Best fit) Derived: The item at head of the priQ may not fit to the current order So you should add a function that returns the best-fit table for that order			

		List Name	Type	Member of class & its declaration
	22	**Busy_No_Share**	**a class derived from PriQueue** *class **Fit_Tables** : public PriQueue<table*> { //add "getBest()" function }	**Restaurant** *Fit_Tables Busy_No_Share;*
		Very important notes		
		- The 3 lists for the tables are objects of the same class as the 3 lists are handled the same way **- Other designs for table lists could be accepted as well if it is justified by the team to their supervisor** **- The order class should be pointing to its assigned resources as follows** **- Each cooking order should point to its assgined chef** **For that reason, no need for "Busy chef" list** **- Each In-service order should point to its assgined (scooter/table)** **For that reason, no need for "Busy scooter" list**		

Reason (Verbal) **(Actual justification should include big O analysis)**			
PriQ: as table are prioritrized by least no. of avail seats (Best fit) Derived: The item at head of the priQ may not fit to the current order So you should add a function that returns the best-fit table for that order			

